



Day 03 — Data Manipulation

The tidyverse (dplyr), loading/storing data, ordering, transforming variables, missing values, merging, restructuring, and aggregation

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-09-02

Topics:

Package management	2
Aggregation: <code>group_by()</code> , <code>mutate()</code> , and <code>summarize()</code>	2
Getting the data	10
Loading and storing data (csv, dta, sav, xlsx)	11
Sums and means of booleans	13
Ordering your data	14
Transforming variables	16
Selecting variables	16
Missing values	20
Restructuring data: <code>pivot_longer()</code> and <code>pivot_wider()</code>	23
Merging data: <code>bind_rows()</code> and joins	24
Try it: Green vote share by Bundestag term	28
Try it: which pollster shows AfD above 20% most often?	29
Try it: rebuild Day 2's polling trend plot	30
Try it: AfD vs. SPD — a negative correlation	31

Package management

Same as every session: bootstrap pacman, then load the tidyverse.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse)
```

Aggregation: `group_by()`, `mutate()`, and `summarize()`

A small toy tibble makes it easy to see exactly what each verb does to every row. The animations below are by [Andrew Heiss](#) (CC BY 4.0) — watch the rows to see what each call actually does.

```
df <- tribble(
  ~cat1, ~cat2, ~x,
  "a",   "j",   1,
  "a",   "j",   2,
  "a",   "k",   3,
  "b",   "j",   4,
  "b",   "k",   5,
  "b",   "k",   6,
  "c",   "j",   7,
  "c",   "j",   8,
  "c",   "k",   9
)
df
```

OUTPUT

```
# A tibble: 9 × 3
  cat1 cat2     x
  <chr> <chr> <dbl>
1 a     j     1
2 a     j     2
3 a     k     3
4 b     j     4
5 b     k     5
6 b     k     6
7 c     j     7
8 c     j     8
9 c     k     9
```

`df()` also happens to be a base R function (the F-distribution density) — naming our tibble `df` shadows it, harmless here since we never need that function, but worth knowing.

mutate(): adds a column, keeps every row

```
df |>
  mutate(y = x - min(x))
```

>_ OUTPUT

```
# A tibble: 9 × 4
  cat1 cat2      x      y
  <chr> <chr> <dbl> <dbl>
1 a     j       1       0
2 a     j       2       1
3 a     k       3       2
4 b     j       4       3
5 b     k       5       4
6 b     k       6       5
7 c     j       7       6
8 c     j       8       7
9 c     k       9       8
```

df		
cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  mutate(
    y = x - min(x)
  )
```

The following works, but doesn't make much sense — an aggregate function like `mean()` inside `mutate()` just gets recycled into every row.

```
df |>
  mutate(avg = mean(x))
```

> OUTPUT

```
# A tibble: 9 × 4
  cat1 cat2      x avg
  <chr> <chr> <dbl> <dbl>
1 a     j      1     5
2 a     j      2     5
3 a     k      3     5
4 b     j      4     5
5 b     k      5     5
6 b     k      6     5
7 c     j      7     5
8 c     j      8     5
9 c     k      9     5
```

summarize(): collapses all rows into one

We instead want `summarize()`, which collapses all rows into one — actual aggregation.

```
df |>
  summarize(avg = mean(x))
```

> OUTPUT

```
# A tibble: 1 × 1
  avg
  <dbl>
1     5
```

Multiple aggregations can be computed at once — this is exactly what the animation below shows.

```
df |>
  summarize(avg = mean(x), total = sum(x), n = n())
```

> OUTPUT

```
# A tibble: 1 × 3
  avg total    n
  <dbl> <dbl> <int>
1     5    45     9
```

df

cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  summarize(
    avg = mean(x),
    total = sum(x),
    n = n()
  )
```

group_by() alone changes nothing visible

It just tags the tibble with grouping metadata; `ungroup()` removes the tag again.

```
df |>
  group_by(cat2)
```

_ OUTPUT

```
# A tibble: 9 × 3
# Groups:   cat2 [2]
  cat1  cat2      x
  <chr> <chr> <dbl>
1 a     j       1
2 a     j       2
3 a     k       3
4 b     j       4
5 b     k       5
6 b     k       6
7 c     j       7
8 c     j       8
9 c     k       9
```

```
df |>
  group_by(cat2) |>
  ungroup()
```

>_ OUTPUT

```
# A tibble: 9 × 3
  cat1  cat2      x
  <chr> <chr> <dbl>
1 a      j      1
2 a      j      2
3 a      k      3
4 b      j      4
5 b      k      5
6 b      k      6
7 c      j      7
8 c      j      8
9 c      k      9
```

df		
cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  group_by(cat2) |>
  ungroup()
```

group_by() + mutate(): still one row per row, computed per group

Same recycling behavior as before, now per group — each row gets its own group's average.

```
df |>
  group_by(cat1) |>
  mutate(avg = mean(x))
```

>_ OUTPUT

```
# A tibble: 9 × 4
# Groups:   cat1 [3]
  cat1 cat2      x    avg
  <chr> <chr> <dbl> <dbl>
1 a     j      1      2
2 a     j      2      2
3 a     k      3      2
4 b     j      4      5
5 b     k      5      5
6 b     k      6      5
7 c     j      7      8
8 c     j      8      8
9 c     k      9      8
```

```
df |>
  group_by(cat1) |>
  mutate(y = x - min(x))
```

>_ OUTPUT

```
# A tibble: 9 × 4
# Groups:   cat1 [3]
  cat1 cat2      x     y
  <chr> <chr> <dbl> <dbl>
1 a     j      1      0
2 a     j      2      1
3 a     k      3      2
4 b     j      4      0
5 b     k      5      1
6 b     k      6      2
7 c     j      7      0
8 c     j      8      1
9 c     k      9      2
```

df

cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  group_by(cat1) |>
  mutate(
    y = x - min(x)
  )
```

group_by() + summarize(): one row per group

```
df |>
  group_by(cat1) |>
  summarize(avg = mean(x), total = sum(x), n = n())
```

> OUTPUT

```
# A tibble: 3 × 4
  cat1    avg total     n
  <chr> <dbl> <dbl> <int>
1 a         2     6     3
2 b         5    15     3
3 c         8    24     3
```

df

cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  group_by(cat1) |>
  summarize(
    avg = mean(x),
    total = sum(x),
    n = n()
  )
```

```
df |>
  group_by(cat2) |>
  summarize(avg = mean(x), total = sum(x), n = n())
```

> OUTPUT

```
# A tibble: 2 × 4
  cat2    avg total     n
  <chr> <dbl> <dbl> <int>
1 j     4.4    22     5
2 k     5.75   23     4
```


df

cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  group_by(cat2) |>
  summarize(
    avg = mean(x),
    total = sum(x),
    n = n()
  )
```

Grouping by two variables collapses to one row per combination — `summarize()` only drops the innermost (last) group, so `cat1` grouping survives and R warns about it; `ungroup()` removes it explicitly.

```
df |>
  group_by(cat1, cat2) |>
  summarize(avg = mean(x), total = sum(x), n = n()) |>
  ungroup()
```

MESSAGE

`summarise()` has regrouped the output.
 i Summaries were computed grouped by cat1 and cat2.
 i Output is grouped by cat1.
 i Use `summarise(.groups = "drop\_last")` to silence this message.
 i Use `summarise(.by = c(cat1, cat2))` for per-operation grouping
 (`?dplyr::dplyr\_by`) instead.

_ OUTPUT

```
# A tibble: 6 × 5
  cat1 cat2   avg total    n
  <chr> <chr> <dbl> <dbl> <int>
1 a     j     1.5     3     2
2 a     k     3       3     1
3 b     j     4       4     1
4 b     k     5.5    11     2
5 c     j     7.5    15     2
6 c     k     9       9     1
```

df		
cat1	cat2	x
a	j	1
a	j	2
a	k	3
b	j	4
b	k	5
b	k	6
c	j	7
c	j	8
c	k	9

```
df |>
  group_by(cat1, cat2) |>
  summarize(
    avg = mean(x),
    total = sum(x),
    n = n()
  ) |>
  ungroup()
```

Getting the data

German federal election polling trend (ZEIT ONLINE), same source as Day 2 — but here we start from the raw wide file (one column per party) instead of the already-reshaped long version, since reshaping is this day's own topic.

```
polls <- read_csv("d03/polls.csv")
```

MESSAGE

Rows: 2077 Columns: 12

— Column specification —

Delimiter: ","

chr (3): Poll_ID, Pollster, Period

dbl (8): n, CDUCSU, SPD, Gruene, FDP, Linke, AfD, BSW

date (1): Date

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
polls |> head(100)
```

> OUTPUT

```
# A tibble: 100 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 90 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

Loading and storing data (csv, dta, sav, xlsx)

`read_csv()` we already know. Other file formats need their own reader — `haven` for Stata/SPSS, `readxl` for Excel. A small sample (15 rows) of the same data, checked in as `polls_sample.<ext>`, so you can see all three side by side without needing the original files.

```
p_load(haven, readxl)
```

```
read_dta("d03/polls_sample.dta") # Stata
```

> OUTPUT

```
# A tibble: 15 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
11 BT02170201S Forsa 2017-02-01 2502 23.01... 35 26 8 6 9
12 BT01170202R Infrates... 2017-02-02 1506 30.01... 34 28 8 6 8
13 BT03170204P Verian 2017-02-04 2233 26.01... 33 29 8 6 8
14 BT02170208T Forsa 2017-02-08 2501 30.01... 34 31 8 5 8
15 BT03170211Q Verian 2017-02-11 1966 02.02... 33 32 7 6 8
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
read_sav("d03/polls_sample.sav") # SPSS
```

> OUTPUT

```
# A tibble: 15 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
11 BT02170201S Forsa 2017-02-01 2502 23.01... 35 26 8 6 9
12 BT01170202R Infrates... 2017-02-02 1506 30.01... 34 28 8 6 8
13 BT03170204P Verian 2017-02-04 2233 26.01... 33 29 8 6 8
14 BT02170208T Forsa 2017-02-08 2501 30.01... 34 31 8 5 8
15 BT03170211Q Verian 2017-02-11 1966 02.02... 33 32 7 6 8
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
read_xlsx("d03/polls_sample.xlsx") # Excel
```

> OUTPUT

```
# A tibble: 15 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP
  <chr> <chr> <dtm> <dbl> <chr> <dbl> <dbl> <dbl> <dbl>
1 BT021701... Forsa 2017-01-04 00:00:00 2001 27.12... 37 21 10 6
2 BT011701... Infrate... 2017-01-05 00:00:00 1505 02.01... 37 20 9 5
3 BT031701... Verian 2017-01-07 00:00:00 834 03.01... 38 22 10 5
4 BT021701... Forsa 2017-01-11 00:00:00 2501 02.01... 37 20 10 6
5 BT031701... Verian 2017-01-14 00:00:00 1878 06.01... 37 21 10 6
6 BT021701... Forsa 2017-01-18 00:00:00 2503 09.01... 38 21 9 6
7 BT031701... Verian 2017-01-21 00:00:00 2805 12.01... 36 21 9 6
8 BT021701... Forsa 2017-01-25 00:00:00 2504 16.01... 37 21 10 6
9 BT011701... Infrate... 2017-01-27 00:00:00 1029 24.01... 35 23 9 6
10 BT031701... Verian 2017-01-28 00:00:00 1996 19.01... 37 23 10 6
11 BT021702... Forsa 2017-02-01 00:00:00 2502 23.01... 35 26 8 6
12 BT011702... Infrate... 2017-02-02 00:00:00 1506 30.01... 34 28 8 6
13 BT031702... Verian 2017-02-04 00:00:00 2233 26.01... 33 29 8 6
14 BT021702... Forsa 2017-02-08 00:00:00 2501 30.01... 34 31 8 5
15 BT031702... Verian 2017-02-11 00:00:00 1966 02.02... 33 32 7 6
# i 3 more variables: Linke <dbl>, AfD <dbl>, BSW <lgl>
```

Excel has no dedicated date type, so `Date` comes back as a full timestamp (`<dtm>`) instead of a plain `<date>` — a small quirk worth knowing about, not a bug.

Storing data works the same way, one `write_*()` per format — `write_csv()`, `write_dta()`, `write_sav()`; Excel needs the `writexl` package. This is how `polls_sample.*` were produced, kept here for reference:

```
p_load(writexl)
polls_sample <- polls |> slice_head(n = 15)
write_dta(polls_sample, "polls_sample.dta")
write_sav(polls_sample, "polls_sample.sav")
write_xlsx(polls_sample, "polls_sample.xlsx")
```

Sums and means of booleans

A logical condition inside `summarize()` is just a column of `TRUE/FALSE` — `sum()` counts the `TRUE`s, `mean()` gives the proportion that are `TRUE`.

```
polls |>
  summarize(
    n_above_20 = sum(SPD > 20)
  )
```

> OUTPUT

```
# A tibble: 1 × 1
  n_above_20
    <int>
1       430
```

Other columns in `polls` do have missing values (e.g. `FDP`, `Linke`), so `na.rm` is worth adding as a habit — same as anywhere else it's needed.

```
sum(1, 2, 3, NA)
```

> OUTPUT

```
[1] NA
```

```
sum(1, 2, 3, NA, na.rm = TRUE)
```

> OUTPUT

```
[1] 6
```

```
polls |>
  summarize(
    n_above_20 = sum(SPD > 20, na.rm = TRUE),
    pct_above_20 = mean(SPD > 20, na.rm = TRUE)
  )
```

> OUTPUT

```
# A tibble: 1 × 2
  n_above_20 pct_above_20
  <int>      <dbl>
1      430      0.207
```

Same idea, grouped — how often each pollster puts AfD above 25% in 2026.

```
polls |>
  filter(Date >= as.Date("2026-01-01")) |>
  group_by(Pollster) |>
  summarize(pct_afd_above_25 = mean(AfD > 25, na.rm = TRUE))
```

> OUTPUT

```
# A tibble: 8 × 2
  Pollster          pct_afd_above_25
  <chr>              <dbl>
1 Allensbach          0.5
2 Forsa              0.667
3 Forschungsgruppe Wahlen 0.727
4 GMS                 0.75
5 INSA                0.855
6 Infratest dimap     0.429
7 Verian              0.375
8 YouGov              0.714
```

Ordering your data

`arrange()` sorts rows; `desc()` reverses the order; extra columns break ties.

```
polls |> arrange(Date)
```

➤ OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date       n Period CDU/CSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa    2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian    2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa    2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian    2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa    2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian    2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa    2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian    2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls |> arrange(desc(Date))
```

➤ OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date       n Period CDU/CSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT02260804Q Forsa    2026-08-04 2501 28.07... 21 12 16 4 12
2 BT05260804K INSA     2026-08-04 2004 31.07... 21 12 13.5 5 11
3 BT05260802J INSA     2026-08-02 1201 27.07... 21 12 14 5 11
4 BT03260731W Verian    2026-07-31 1456 22.07... 21 13 15 4 12
5 BT06260731K Forschun... 2026-07-31 1405 28.07... 23 12 13 4 12
6 BT02260728P Forsa    2026-07-28 2502 21.07... 21 12 16 4 13
7 BT05260728I INSA     2026-07-28 2006 24.07... 21 12.5 13.5 4 11
8 BT05260724H INSA     2026-07-24 1204 20.07... 21 12 14 4 11
9 BT09260723C Allensba... 2026-07-23 1040 03.07... 24 13 13 5.5 10
10 BT022607210 Forsa    2026-07-21 2502 14.07... 22 12 16 4 12
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls |> arrange(Pollster, Date)
```

➤ OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date       n Period CDU/CSU SPD Gruene FDP Linke
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT09171025X Allensba... 2017-10-25 1454 07.10... 33 20.5 9.5 12 9
2 BT09171130Y Allensba... 2017-11-30 1299 22.11... 33 21.5 11 10 9
3 BT09171221A Allensba... 2017-12-21 1443 01.12... 34 21 11.5 10 9
4 BT09180125B Allensba... 2018-01-25 1221 05.01... 34 21 10.5 10 8.5
5 BT09180223C Allensba... 2018-02-23 1271 02.02... 32 17.5 12 11 9.5
6 BT09180321D Allensba... 2018-03-21 1256 02.03... 34 19 11 10 10
7 BT09180425E Allensba... 2018-04-25 1279 03.04... 34 20.5 11 10 9
8 BT09180517F Allensba... 2018-05-17 1253 24.04... 34 20 11 9 10
9 BT09180620G Allensba... 2018-06-20 1279 01.06... 33 21 11 9.5 9.5
10 BT09180717H Allensba... 2018-07-17 1295 01.07... 30.5 20 12 9.5 9
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

Transforming variables

`rename()` just relabels a column.

```
polls |>
  rename(Union = CDU/CSU)
```

➤ OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date       n Period Union SPD Gruene FDP Linke AfD
  <chr>    <chr>   <date>   <dbl> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT0217... Forsa    2017-01-04 2001 27.12... 37 21 10 6 9 12
2 BT0117... Infrate... 2017-01-05 1505 02.01... 37 20 9 5 9 15
3 BT0317... Verian   2017-01-07 834 03.01... 38 22 10 5 9 12
4 BT0217... Forsa    2017-01-11 2501 02.01... 37 20 10 6 9 12
5 BT0317... Verian   2017-01-14 1878 06.01... 37 21 10 6 10 12
6 BT0217... Forsa    2017-01-18 2503 09.01... 38 21 9 6 9 11
7 BT0317... Verian   2017-01-21 2805 12.01... 36 21 9 6 11 12
8 BT0217... Forsa    2017-01-25 2504 16.01... 37 21 10 6 9 12
9 BT0117... Infrate... 2017-01-27 1029 24.01... 35 23 9 6 8 14
10 BT0317... Verian   2017-01-28 1996 19.01... 37 23 10 6 10 11
# i 2,067 more rows
# i 1 more variable: BSW <dbl>
```

Selecting variables

`select()` takes a subset of columns, and can also be used to drop specific columns.


```
polls |>
  select(Pollster, Date, Period)
```

➤ OUTPUT

```
# A tibble: 2,077 × 3
  Pollster      Date      Period
  <chr>         <date>    <chr>
1 Forsa        2017-01-04 27.12.-30.12.
2 Infratest dimap 2017-01-05 02.01.-04.01.
3 Verian       2017-01-07 03.01.-05.01.
4 Forsa        2017-01-11 02.01.-06.01.
5 Verian       2017-01-14 06.01.-11.01.
6 Forsa        2017-01-18 09.01.-13.01.
7 Verian       2017-01-21 12.01.-18.01.
8 Forsa        2017-01-25 16.01.-20.01.
9 Infratest dimap 2017-01-27 24.01.-25.01.
10 Verian      2017-01-28 19.01.-25.01.
# i 2,067 more rows
```

```
polls |>
  select(-BSW)
```

➤ OUTPUT

```
# A tibble: 2,077 × 11
  Poll_ID Pollster Date      n Period CDU/CSU SPD Gruene FDP Linke
  <chr>    <chr>    <date>    <dbl> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa    2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian    2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa    2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian    2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa    2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian    2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa    2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian    2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 1 more variable: AfD <dbl>
```

`case_when()` recodes a variable into categories, checked top to bottom — the first matching condition wins. Here: which Bundestag term a poll falls into, from its date.

```
polls_with_terms <- polls |>
  mutate(
    term = case_when(
      Date < as.Date("2021-10-26") ~ "19. Wahlperiode",
      Date < as.Date("2025-03-25") ~ "20. Wahlperiode",
      .default = "21. Wahlperiode"
    )
  )
polls_with_terms
```

>_ OUTPUT

```
# A tibble: 2,077 × 13
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 3 more variables: AfD <dbl>, BSW <dbl>, term <chr>
```

```
# unique combinations of pollster and term
polls_with_terms |> distinct(Pollster, term)
```

>_ OUTPUT

```
# A tibble: 24 × 2
  Pollster term
  <chr> <chr>
1 Forsa 19. Wahlperiode
2 Infratest dimap 19. Wahlperiode
3 Verian 19. Wahlperiode
4 YouGov 19. Wahlperiode
5 Forschungsgruppe Wahlen 19. Wahlperiode
6 GMS 19. Wahlperiode
7 Allensbach 19. Wahlperiode
8 INSA 19. Wahlperiode
9 Forsa 20. Wahlperiode
10 Infratest dimap 20. Wahlperiode
# i 14 more rows
```

In how many unique terms has each pollster been active? (In fact, the same for all of them.)

```
polls_with_terms |>
  distinct(term, Pollster) |>
  group_by(Pollster) |>
  summarize(terms = n())
```

> OUTPUT

```
# A tibble: 8 × 2
  Pollster      terms
  <chr>        <int>
1 Allensbach      3
2 Forsa           3
3 Forschungsgruppe Wahlen 3
4 GMS             3
5 INSA            3
6 Infratest dimap  3
7 Verian          3
8 YouGov          3
```

But what share of *all* polls in a given term did each pollster carry out?

```
polls_with_terms |>
  group_by(term, Pollster) |>
  summarize(polls = n()) |>
  group_by(term) |>
  mutate(percentage = polls / sum(polls), sum_polls = sum(polls))
```

MESSAGE

```
`summarise()` has regrouped the output.
i Summaries were computed grouped by term and Pollster.
i Output is grouped by term.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(term, Pollster))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

> OUTPUT

```
# A tibble: 24 × 5
# Groups:   term [3]
  term      Pollster      polls percentage sum_polls
  <chr>      <chr>        <int>      <dbl>      <int>
1 19. Wahlperiode Allensbach      52    0.0607      856
2 19. Wahlperiode Forsa         258    0.301      856
3 19. Wahlperiode Forschungsgruppe Wahlen 79    0.0923      856
4 19. Wahlperiode GMS           36    0.0421      856
5 19. Wahlperiode INSA           8    0.00935     856
6 19. Wahlperiode Infratest dimap 116    0.136      856
7 19. Wahlperiode Verian        242    0.283      856
8 19. Wahlperiode YouGov         65    0.0759      856
9 20. Wahlperiode Allensbach      41    0.0452     908
10 20. Wahlperiode Forsa        173    0.191     908
# i 14 more rows
```

Missing values

BSW only became a party in 2024 — every poll before that has `share = NA`.

```
polls |> filter(is.na(BSW)) |> nrow() # how many rows have NAs?
```

> OUTPUT

```
[1] 1511
```

```
polls |> filter(!is.na(BSW)) # only polls that actually asked about BSW
```

> OUTPUT

```
# A tibble: 566 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT06240112G Forschun... 2024-01-12 1337 09.01... 31 13 14 4 4
2 BT05240120V INSA 2024-01-20 1203 15.01... 30 13 13 5 3
3 BT02240123K Forsa 2024-01-23 2503 16.01... 31 14 14 4 4
4 BT03240127P Verian 2024-01-27 1922 16.01... 29 16 16 4 4
5 BT05240127X INSA 2024-01-27 1201 22.01... 31 14 13 4 4
6 BT05240129Y INSA 2024-01-29 2002 26.01... 30 14.5 12.5 5 3.5
7 BT02240130L Forsa 2024-01-30 2506 23.01... 32 15 14 3 3
8 BT01240201B Infrates... 2024-02-01 1303 29.01... 30 16 14 4 3
9 BT06240202H Forschun... 2024-02-02 1217 30.01... 31 15 13 4 3
10 BT05240203A INSA 2024-02-03 1202 29.01... 30 15 13 4 4
# i 556 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

```
polls |> drop_na(BSW) # same idea, tidyr's dedicated verb
```

> OUTPUT

```
# A tibble: 566 × 12
  Poll_ID Pollster Date n Period CDU CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT06240112G Forschun... 2024-01-12 1337 09.01... 31 13 14 4 4
2 BT05240120V INSA 2024-01-20 1203 15.01... 30 13 13 5 3
3 BT02240123K Forsa 2024-01-23 2503 16.01... 31 14 14 4 4
4 BT03240127P Verian 2024-01-27 1922 16.01... 29 16 16 4 4
5 BT05240127X INSA 2024-01-27 1201 22.01... 31 14 13 4 4
6 BT05240129Y INSA 2024-01-29 2002 26.01... 30 14.5 12.5 5 3.5
7 BT02240130L Forsa 2024-01-30 2506 23.01... 32 15 14 3 3
8 BT01240201B Infrates... 2024-02-01 1303 29.01... 30 16 14 4 3
9 BT06240202H Forschun... 2024-02-02 1217 30.01... 31 15 13 4 3
10 BT05240203A INSA 2024-02-03 1202 29.01... 30 15 13 4 4
# i 556 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

Treating “not asked yet” as “0%” can be the better modeling/data-handling choice — but make it explicit, rather than letting NAs silently vanish.

```
polls |> mutate(BSW = replace_na(BSW, 0))
```

> OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date n Period CDU/SU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

`coalesce()` picks the first non-missing value across several columns, left to right — handy when the same information was collected repeatedly and you just want whichever answer came in.

```
survey_waves <- tribble(
  ~Respondent_ID, ~Response_May, ~Response_June, ~Response_July,
  1, "CDUCSU", NA, "Gruene",
  2, NA, "FDP", NA,
  3, NA, NA, "AfD",
  4, NA, NA, NA
)
survey_waves |>
  mutate(Latest_Response = coalesce(Response_July, Response_June, Response_May))
```

> OUTPUT

```
# A tibble: 4 × 5
  Respondent_ID Response_May Response_June Response_July Latest_Response
  <dbl> <chr> <chr> <chr> <chr>
1 1 CDUCSU <NA> Gruene Gruene
2 2 <NA> FDP <NA> FDP
3 3 <NA> <NA> AfD AfD
4 4 <NA> <NA> <NA> <NA>
```

Real surveys often encode different *kinds* of missingness as sentinel numbers instead of leaving the cell blank. Same shape and entries as `survey_waves` above, but every non-response was coded with a number instead: **97** means “still undecided” (an actual answer worth keeping), **98/99** mean “don’t know”/“refused” (true missingness).

```
survey_sentinels <- tribble(
  ~Respondent_ID, ~Response_May, ~Response_June, ~Response_July,
  1, "CDUCSU", "99", "Gruene",
  2, "98", "FDP", "97",
  3, "97", "98", "AfD",
  4, "99", "97", "98"
)
```

`~ .x` is shorthand for a one-off `function(x) ...` — anonymous functions are covered properly on Day 4, previewed here since `across()` needs one to apply the same recoding to several columns at once. `replace_values()` only touches the values named on the left of each formula — everything else passes through unchanged.

```
survey_sentinels |>
  mutate(
    across(
      starts_with("Response"), ~ replace_values(
        .x,
        c("98", "99") ~ NA,
        "97" ~ "Undecided"
      )
    )
  )
```

> OUTPUT

```
# A tibble: 4 × 4
  Respondent_ID Response_May Response_June Response_July
      <dbl> <chr>          <chr>          <chr>
1           1 CDUCSU      <NA>          Gruene
2           2 <NA>          FDP            Undecided
3           3 Undecided <NA>          AfD
4           4 <NA>          Undecided      <NA>
```

Needing `across()` to mutate several look-alike columns the same way is itself a hint that the data would rather be long — one column, one `mutate()`, no `across()/starts_with()` needed at all.

```
survey_sentinels |>
  pivot_longer(starts_with("Response"), names_to = "wave", values_to = "response") |>
  mutate(response = replace_values(response, c("98", "99") ~ NA, "97" ~ "Undecided"))
```

>_ OUTPUT

```
# A tibble: 12 × 3
  Respondent_ID wave      response
      <dbl> <chr>      <chr>
1           1 Response_May CDUCSU
2           1 Response_June <NA>
3           1 Response_July Gruene
4           2 Response_May <NA>
5           2 Response_June FDP
6           2 Response_July Undecided
7           3 Response_May Undecided
8           3 Response_June <NA>
9           3 Response_July AfD
10          4 Response_May <NA>
11          4 Response_June Undecided
12          4 Response_July <NA>
```

Restructuring data: `pivot_longer()` and `pivot_wider()`

One row per poll x party (long) instead of one row per poll (wide) — this is the reshaping step Day 2's `polls.csv` already came pre-made from. Long format is what most tidyverse tools actually want: `ggplot2` needs one column to map color/fill to (Day 2's party colors), `group_by()/summarize()` need the grouping variable to be an actual column rather than spread across many columns, and joins match on a single key column the same way.

```
polls_long <- polls |>
  pivot_longer(CDUCSU:BSW, names_to = "party", values_to = "share")
polls_long
```

>_ OUTPUT

```
# A tibble: 14,539 × 7
  Poll_ID Pollster Date      n Period      party share
  <chr>    <chr>   <date>  <dbl> <chr>      <chr>  <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. CDUCSU 37
2 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. SPD 21
3 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. Gruene 10
4 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. FDP 6
5 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. Linke 9
6 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. AfD 12
7 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. BSW NA
8 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. CDUCSU 37
9 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. SPD 20
10 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. Gruene 9
# i 14,529 more rows
```

`pivot_wider()` is the mirror image — back to one row per poll, the same idea Day 2 used for the AfD-vs-SPD correlation plot.

```
polls_long |>
  pivot_wider(names_from = party, values_from = share)
```

> OUTPUT

```
# A tibble: 2,077 × 12
  Poll_ID Pollster Date n Period CDU/CSU SPD Gruene FDP Linke
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12... 37 21 10 6 9
2 BT01170105P Infrates... 2017-01-05 1505 02.01... 37 20 9 5 9
3 BT03170107L Verian 2017-01-07 834 03.01... 38 22 10 5 9
4 BT02170111P Forsa 2017-01-11 2501 02.01... 37 20 10 6 9
5 BT03170114M Verian 2017-01-14 1878 06.01... 37 21 10 6 10
6 BT02170118Q Forsa 2017-01-18 2503 09.01... 38 21 9 6 9
7 BT03170121N Verian 2017-01-21 2805 12.01... 36 21 9 6 11
8 BT02170125R Forsa 2017-01-25 2504 16.01... 37 21 10 6 9
9 BT01170127Q Infrates... 2017-01-27 1029 24.01... 35 23 9 6 8
10 BT031701280 Verian 2017-01-28 1996 19.01... 37 23 10 6 10
# i 2,067 more rows
# i 2 more variables: AfD <dbl>, BSW <dbl>
```

Merging data: `bind_rows()` and joins

`bind_rows()` stacks tibbles with the same columns on top of each other. Filtering by party is only possible now that party is a column (long format) — in the wide file, each row already holds every party at once.

```
afd_polls <- polls_long |> filter(party == "AfD") |> head(n = 3)
afd_polls
```

> OUTPUT

```
# A tibble: 3 × 7
  Poll_ID Pollster Date n Period party share
  <chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.–30.12. AfD 12
2 BT01170105P Infratest dimap 2017-01-05 1505 02.01.–04.01. AfD 15
3 BT03170107L Verian 2017-01-07 834 03.01.–05.01. AfD 12
```



```
cdu_polls <- polls_long |> filter(party == "CDUCSU") |> slice_head(n = 3)
cdu_polls
```

> OUTPUT

```
# A tibble: 3 × 7
  Poll_ID Pollster Date n Period party share
  <chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.–30.12. CDUCSU 37
2 BT01170105P Infratest dimap 2017-01-05 1505 02.01.–04.01. CDUCSU 37
3 BT03170107L Verian 2017-01-07 834 03.01.–05.01. CDUCSU 38
```

```
bind_rows(afd_polls, cdu_polls)
```

> OUTPUT

```
# A tibble: 6 × 7
  Poll_ID Pollster Date n Period party share
  <chr> <chr> <date> <dbl> <chr> <chr> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.–30.12. AfD 12
2 BT01170105P Infratest dimap 2017-01-05 1505 02.01.–04.01. AfD 15
3 BT03170107L Verian 2017-01-07 834 03.01.–05.01. AfD 12
4 BT021701040 Forsa 2017-01-04 2001 27.12.–30.12. CDUCSU 37
5 BT01170105P Infratest dimap 2017-01-05 1505 02.01.–04.01. CDUCSU 37
6 BT03170107L Verian 2017-01-07 834 03.01.–05.01. CDUCSU 38
```

Joins add columns from a second table by matching a key — here, each party’s group in the European Parliament (2024–2029 term). Two mismatches on purpose: BSW isn’t in `party_ep_groups` (no match on the left), and Die PARTEI (a real party, one EP seat, “Non-attached” — not tracked by the Bundestag polling data) isn’t in `polls_long` (no match on the right), so all four join types actually differ from each other.

```

party_ep_groups <- tribble(
  ~party, ~ep_group,
  "CDUCSU", "EPP",
  "SPD", "S&D",
  "Gruene", "Greens/EFA",
  "FDP", "Renew Europe",
  "Linke", "The Left",
  "AfD", "ESN",
  "DIE PARTEI", "Non-attached"
  # no BSW
)

# just the most recent polling day (2 polls x 7 parties = 14 rows) – small
# enough to actually see every join's differences, not just row counts
recent_polls_long <- polls_long |> filter(Date == max(Date))
recent_polls_long

```

> OUTPUT

```

# A tibble: 14 × 7
  Poll_ID Pollster Date      n Period party share
  <chr>    <chr>   <date>   <dbl> <chr>   <chr> <dbl>
1 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. CDU/CSU 21
2 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. SPD 12
3 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. Gruene 16
4 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. FDP 4
5 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. Linke 12
6 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. AfD 28
7 BT02260804Q Forsa 2026-08-04 2501 28.07.–03.08. BSW NA
8 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. CDU/CSU 21
9 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. SPD 12
10 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. Gruene 13.5
11 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. FDP 5
12 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. Linke 11
13 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. AfD 28
14 BT05260804K INSA 2026-08-04 2004 31.07.–03.08. BSW 4

```

`left_join()` keeps every row of the left table (`recent_polls_long`), filling in `NA` for any party without a match — BSW rows kept, `ep_group = NA`; DIE PARTEI never appears, since it isn't a row of `recent_polls_long` to begin with.

```
recent_polls_long |> left_join(party_ep_groups, by = "party")
```

➤ OUTPUT

```
# A tibble: 14 × 8
  Poll_ID Pollster Date      n Period      party share ep_group
  <chr>    <chr>   <date>   <dbl> <chr>      <chr>   <dbl> <chr>
1 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. CDU/CSU 21 EPP
2 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. SPD 12 S&D
3 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Gruene 16 Greens/EFA
4 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. FDP 4 Renew Europe
5 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Linke 12 The Left
6 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. AfD 28 ESN
7 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. BSW NA <NA>
8 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. CDU/CSU 21 EPP
9 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. SPD 12 S&D
10 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Gruene 13.5 Greens/EFA
11 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. FDP 5 Renew Europe
12 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Linke 11 The Left
13 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. AfD 28 ESN
14 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. BSW 4 <NA>
```

`inner_join()` keeps only rows that match on both sides — BSW rows dropped entirely, DIE PARTEI still never appears.

```
recent_polls_long |> inner_join(party_ep_groups, by = "party")
```

➤ OUTPUT

```
# A tibble: 12 × 8
  Poll_ID Pollster Date      n Period      party share ep_group
  <chr>    <chr>   <date>   <dbl> <chr>      <chr>   <dbl> <chr>
1 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. CDU/CSU 21 EPP
2 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. SPD 12 S&D
3 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Gruene 16 Greens/EFA
4 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. FDP 4 Renew Europe
5 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Linke 12 The Left
6 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. AfD 28 ESN
7 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. CDU/CSU 21 EPP
8 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. SPD 12 S&D
9 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Gruene 13.5 Greens/EFA
10 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. FDP 5 Renew Europe
11 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Linke 11 The Left
12 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. AfD 28 ESN
```

`right_join()` keeps every row of `party_ep_groups` (the right table) — same BSW-dropping as `inner_join()`, but now DIE PARTEI shows up once, as a row with `NA` for every `recent_polls_long` column.

```
recent_polls_long |> right_join(party_ep_groups, by = "party")
```

➤ OUTPUT

```
# A tibble: 13 × 8
  Poll_ID Pollster Date      n Period      party      share ep_group
  <chr>    <chr>   <date>   <dbl> <chr>      <chr>    <dbl> <chr>
1 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. CDU/CSU    21 EPP
2 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. SPD      12 S&D
3 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Gruene    16 Greens/...
4 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. FDP       4 Renew E...
5 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Linke    12 The Left
6 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. AfD     28 ESN
7 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. CDU/CSU    21 EPP
8 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. SPD      12 S&D
9 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Gruene   13.5 Greens/...
10 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. FDP       5 Renew E...
11 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Linke    11 The Left
12 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. AfD     28 ESN
13 <NA>      <NA>    NA      NA <NA>      DIE PARTEI NA Non-att...
```

`full_join()` keeps everything from both sides — BSW rows kept with `ep_group = NA` (like `left_join()`), plus that same all-NA DIE PARTEI row (like `right_join()`).

```
recent_polls_long |> full_join(party_ep_groups, by = "party")
```

➤ OUTPUT

```
# A tibble: 15 × 8
  Poll_ID Pollster Date      n Period      party      share ep_group
  <chr>    <chr>   <date>   <dbl> <chr>      <chr>    <dbl> <chr>
1 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. CDU/CSU    21 EPP
2 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. SPD      12 S&D
3 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Gruene    16 Greens/...
4 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. FDP       4 Renew E...
5 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. Linke    12 The Left
6 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. AfD     28 ESN
7 BT02260804Q Forsa 2026-08-04 2501 28.07.-03.08. BSW      NA <NA>
8 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. CDU/CSU    21 EPP
9 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. SPD      12 S&D
10 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Gruene   13.5 Greens/...
11 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. FDP       5 Renew E...
12 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. Linke    11 The Left
13 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. AfD     28 ESN
14 BT05260804K INSA 2026-08-04 2004 31.07.-03.08. BSW       4 <NA>
15 <NA>      <NA>    NA      NA <NA>      DIE PARTEI NA Non-att...
```

Try it: Green vote share by Bundestag term

Your task: compute the Greens' average polled vote share within each Bundestag term (Wahlperiode), ranked from highest to lowest — starting fresh from `polls`, not the `term/long` objects already built above.

```
polls |>
  pivot_longer(CDUCSU:BSW, names_to = "party", values_to = "share") |>
  mutate(
    term = case_when(
      Date < as.Date("2021-10-26") ~ "19. Wahlperiode",
      Date < as.Date("2025-03-25") ~ "20. Wahlperiode",
      .default = "21. Wahlperiode"
    )
  ) |>
  filter(party == "Gruene") |>
  group_by(term) |>
  summarize(avg_share = mean(share, na.rm = TRUE)) |>
  arrange(desc(avg_share))
```

> OUTPUT

```
# A tibble: 3 × 2
  term      avg_share
<chr>      <dbl>
1 19. Wahlperiode    16.7
2 20. Wahlperiode    15.6
3 21. Wahlperiode    12.2
```

Try it: which pollster shows AfD above 20% most often?

Your task: starting fresh from `polls` again, work out what share of each pollster's polls put AfD above 20%, ranked from most to least often.

```
polls |>
  pivot_longer(CDUCSU:BSW, names_to = "party", values_to = "share") |>
  filter(party == "AfD") |>
  group_by(Pollster) |>
  summarize(pct_above_20 = mean(share > 20, na.rm = TRUE)) |>
  arrange(desc(pct_above_20))
```

> OUTPUT

```
# A tibble: 8 × 2
  Pollster      pct_above_20
<chr>          <dbl>
1 INSA          0.451
2 YouGov        0.227
3 Forschungsgruppe Wahlen 0.216
4 GMS           0.192
5 Forsa         0.188
6 Allensbach    0.162
7 Infratest dimap 0.146
8 Verian        0.0727
```

Try it: rebuild Day 2's polling trend plot

Your task: rebuild Day 2's "color scales" plot, but starting straight from the wide `polls` tibble instead of an already-long one — there's no one column called `party` to hand to `color =` here, only 7 separate share columns.

```
party_colors <- c(
  CDU = "#000000",
  SPD = "#E3000F",
  Gruene = "#46962b",
  FDP = "#ffed00",
  Linke = "#be3075",
  AfD = "#009ee0",
  BSW = "#7d3f98"
)

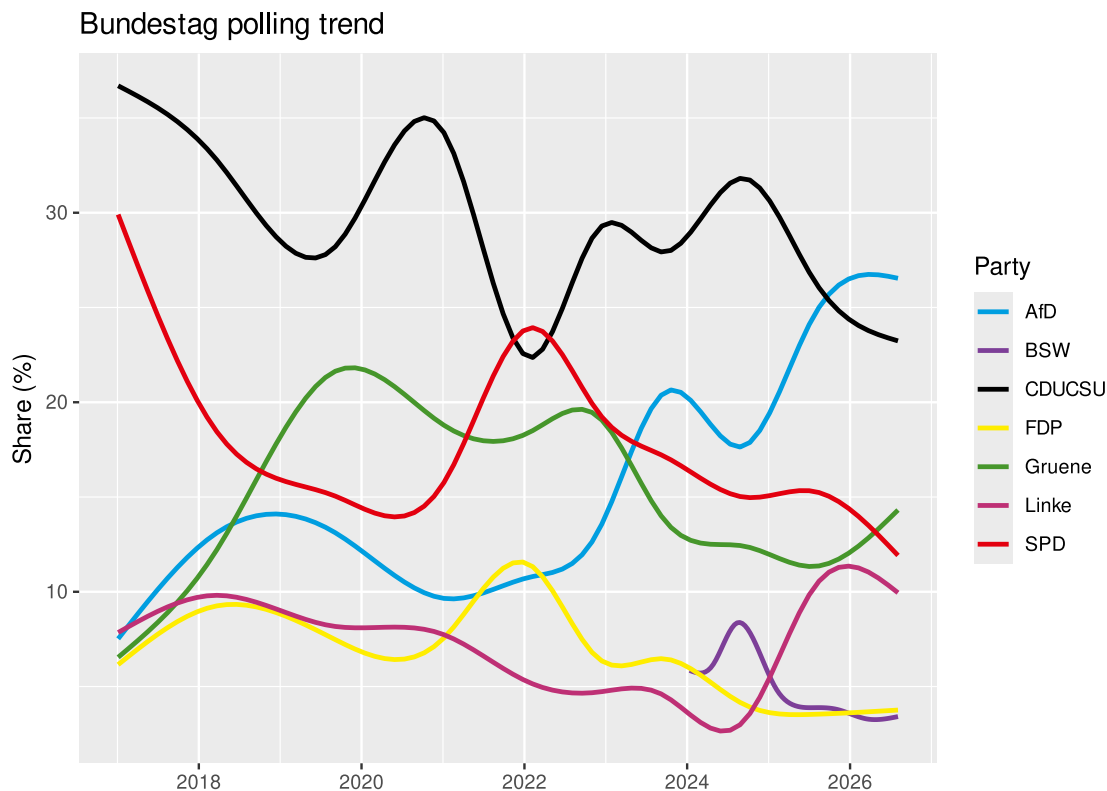
polls |>
  pivot_longer(CDU:BSW, names_to = "party", values_to = "share") |>
  ggplot(aes(x = Date, y = share, color = party)) +
  geom_smooth(se = FALSE) +
  scale_color_manual(values = party_colors) +
  labs(title = "Bundestag polling trend", x = NULL, y = "Share (%)", color = "Party")
```

MESSAGE

``geom_smooth()`` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'

WARNING

Warning: Removed 1548 rows containing non-finite outside the scale range (``stat_smooth()``).



Try it: AfD vs. SPD — a negative correlation

Your task: check whether AfD and SPD support move in opposite directions — a Day 2 question, worth revisiting now that `pivot_wider()` is fresh. Pair each poll's AfD and SPD share into one row per poll, then plot one against the other, overall and split out by year.

```
afd_spd <- polls_long |>
  filter(party %in% c("AfD", "SPD")) |>
  pivot_wider(names_from = party, values_from = share) |>
  mutate(year = year(Date))
afd_spd
```

> OUTPUT

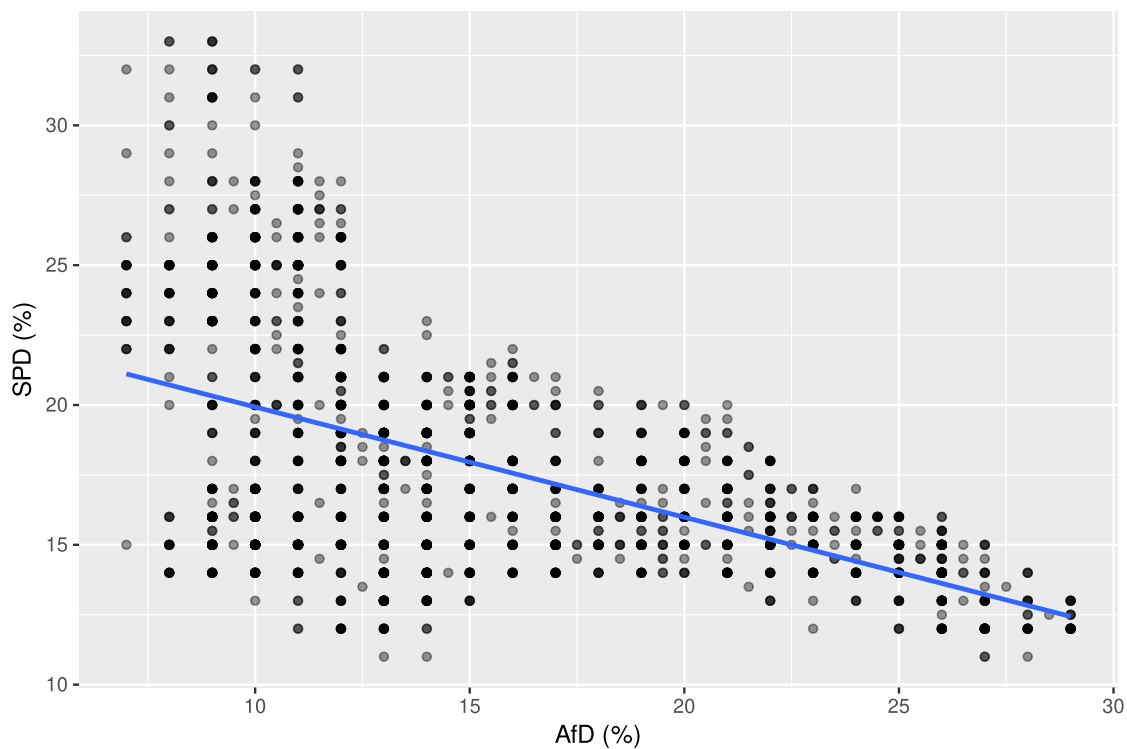
```
# A tibble: 2,077 × 8
  Poll_ID Pollster Date n Period SPD AfD year
  <chr> <chr> <date> <dbl> <chr> <dbl> <dbl> <dbl>
1 BT021701040 Forsa 2017-01-04 2001 27.12.-30.12. 21 12 2017
2 BT01170105P Infratest dimap 2017-01-05 1505 02.01.-04.01. 20 15 2017
3 BT03170107L Verian 2017-01-07 834 03.01.-05.01. 22 12 2017
4 BT02170111P Forsa 2017-01-11 2501 02.01.-06.01. 20 12 2017
5 BT03170114M Verian 2017-01-14 1878 06.01.-11.01. 21 12 2017
6 BT02170118Q Forsa 2017-01-18 2503 09.01.-13.01. 21 11 2017
7 BT03170121N Verian 2017-01-21 2805 12.01.-18.01. 21 12 2017
8 BT02170125R Forsa 2017-01-25 2504 16.01.-20.01. 21 12 2017
9 BT01170127Q Infratest dimap 2017-01-27 1029 24.01.-25.01. 23 14 2017
10 BT031701280 Verian 2017-01-28 1996 19.01.-25.01. 23 11 2017
# i 2,067 more rows
```

```
afd_spd |>
  ggplot(aes(x = AfD, y = SPD)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "AfD vs. SPD polling: a negative correlation", x = "AfD (%)", y =
"SPD (%)")
```

MESSAGE

`geom\_smooth()` using formula = 'y ~ x'

AfD vs. SPD polling: a negative correlation



```
afd_spd |>
  ggplot(aes(x = AfD, y = SPD)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(~year) +
  labs(title = "AfD vs. SPD polling: a negative correlation over the years", x = "AfD
(%)", y = "SPD (%)")
```

MESSAGE

`geom\_smooth()` using formula = 'y ~ x'

AfD vs. SPD polling: a negative correlation over the years

